Integrity Constraints

- Integrity constraints in a Database Management System are **rules** that help keep the data in a database accurate, consistent and reliable.
- They act like **a set of guidelines** that ensure all the information stored in the database follows specific standards.

The picture can't be displayed.

1. Domain Constraints

- Domain constraints are a type of integrity constraint that ensure the **values stored in a column of a database are valid** and within a specific range or domain.
- They define what type of data is allowed in a column and restrict invalid data entry.
- The data type of domain include string, char, time, integer, date, currency etc.

Student_Id	Name	Semester	Age
21CSE100	Aniket Kumar	6th	20
21CSE101	Shashwat Dubey	7th	21
21CSE102	Manvendra Sharma	8th	22
21CSE103	Ashmit Dubey	5th	20

- Student_Id: Must be unique and follow a specific format like 21CSE###. No duplicates or invalid formats allowed.

- Name: Accepts only valid text (no numbers) and cannot be left empty (NOT NULL constraint).

- Semester: Allows specific values like 5th, 6th, etc., and ensures valid input (e.g., no 10th if not permitted).

- Age: Must be an integer within a reasonable range (e.g., 18-30) and cannot contain invalid data like negative numbers or text.

- Types of Domain Constraints:

- **NOT NULL Constraint**: Ensures No records can have NULL value.
- **CHECK Constraint**: This Constraint Checks for any specified condition over any attribute.
- Why Domain Constraints Are Important :

- They **prevent invalid or inconsistent data** from entering the database.
- They ensure the database is **reliable and follows predefined business rules**.
- They make the database **easier to manage** and maintain by **reducing errors**.

2. Entity Integrity Constraints

- Entity integrity constraints state that primary key can never contain null value because primary key is used to determine individual rows in a relation uniquely, if primary key contains null value then we cannot identify those rows. A table can contain null value in it except primary key field.

- **Key Features of Entity Integrity Constraints:**

- **Uniqueness**: The primary key value must be unique for each row in the table. No duplicate entries are allowed in the primary key column.
- **NOT NULL**: The primary key column cannot contain NULL values, as every row must have a valid identifier.
- **Essential for Table Design**: Ensures that every record in the table can be uniquely identified, preventing ambiguity.

3. Key Constraints

- Why Key Constraints Are Important ?

- **Prevent Duplicates**: Ensure unique identification of rows.
- **Maintain Relationships:** Enable proper linking between tables (via foreign keys).
- **Enforce Data Integrity**: Prevent invalid or inconsistent data.
- Key constraints ensure that certain columns or combinations of columns in a table uniquely identify each row.
- These rules are essential for maintaining data integrity and preventing duplicate or ambiguous records.

3.1 Primary Key Constraints

- It states that the primary key attributes are required to be unique and not null.
- That is, primary key attributes of a relation must not have null values and primary key attributes of two tuples must never be same.
- This constraint is specified on database schema to the primary key attributes to ensure that no two tuples are same.

3.2 Unique Key Constraints

- The Unique key constraint in DBMS ensures that all values in a specified column (or group of columns) are distinct across the table.
- It prevents duplicate entries, maintaining data integrity, but unlike the primary key, it allows one NULL value.

4. Referential integrity constraints

- Referential integrity constraints are rules that ensure relationships between tables remain consistent.
- They enforce that a foreign key in one table must either match a value in the referenced primary key of another table.
- This guarantees the logical connection between related tables in a relational database.

- Why Referential Integrity Constraints Are Important ?

- Maintains Consistency: Ensures relationships between tables are valid.
- Prevents Orphan Records: Avoids cases where a record in a child table references a non-existent parent record.
- Enforces Logical Relationships: Strengthens the logical structure of a relational database.

5. Assertion

- An assertion is a declarative mechanism in a database that ensures a specific condition or rule is always satisfied across the entire database.
- It is a global integrity constraint, meaning it applies to multiple tables or the entire database rather than being limited to a single table or column.
- When an assertion is made, the system tests it for validity. This testing may introduce a significant amount of overhead; hence assertions should be used with great care.

6. Triggers

- A trigger is a procedural statement in a database that is automatically executed in response to certain events such as INSERT, UPDATE, or DELETE.
- Triggers are often used to enforce complex integrity constraints or implement business rules that cannot be captured using standard constraints like primary keys or foreign keys.

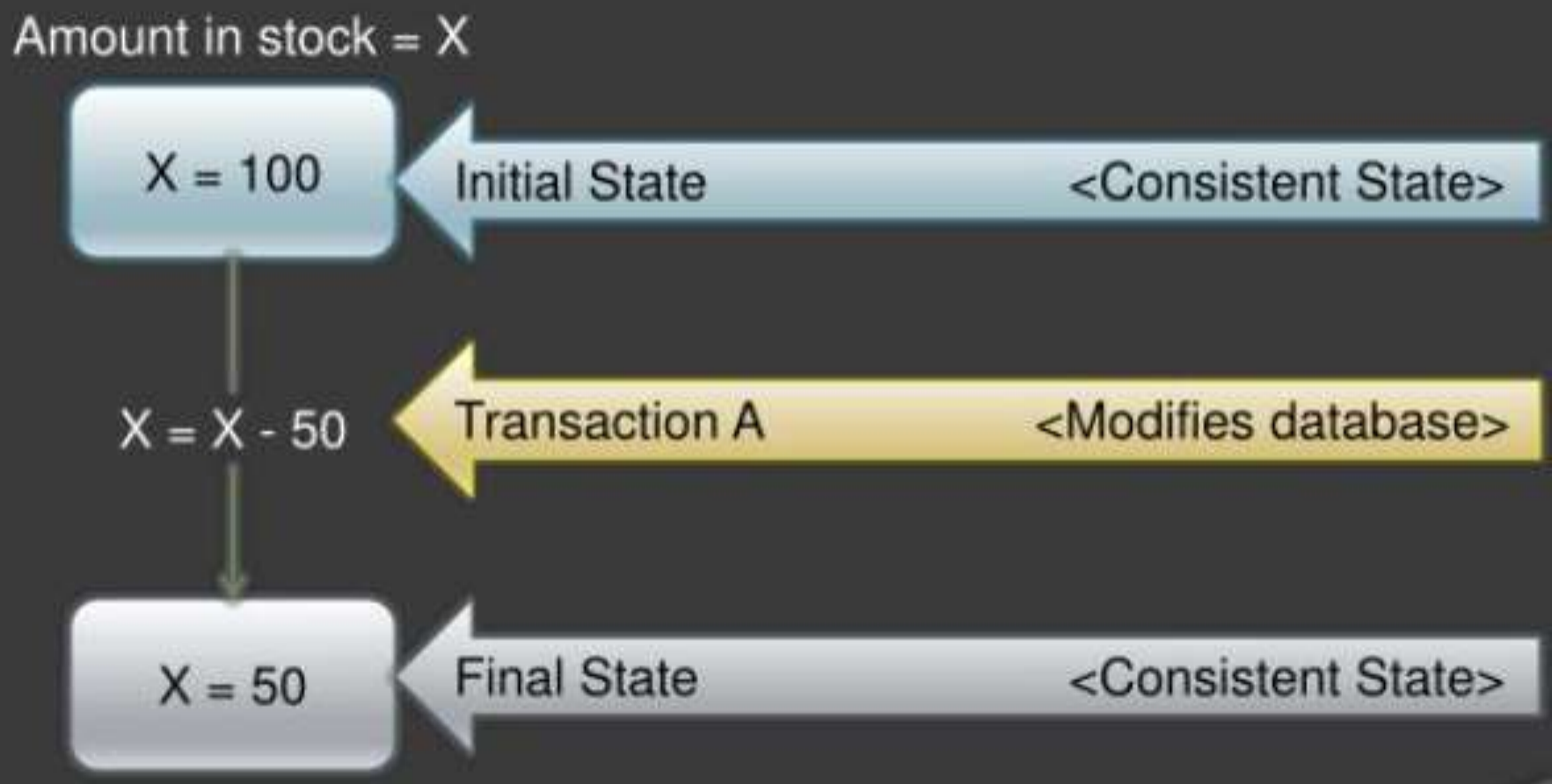
Recovery & Concurrency Control

TRANSACTION PROCESSING CONCEPTS

What is a Transaction?

- A transaction is a logical unit of work that must be either entirely completed or aborted.
- A database request is the equivalent of a single SQL statement in an application program or transaction.
- A transaction that changes the contents of the database must alter the database from one consistent database state to another.
- To ensure consistency of the database, every transaction must begin with the database in a known consistent state.

Example of Transaction



Transaction ACID Properties

Atomic

- "ALL OR NOTHING"
- All statements in the transaction are treated as a unit.
- Transaction cannot be subdivided
- If the transaction completes successfully, everything is committed
- If the transaction fails, everything done up to the point of failure is rolled back.

Consistent

Transaction → transform database from one consistent state to another consistent state

ACID

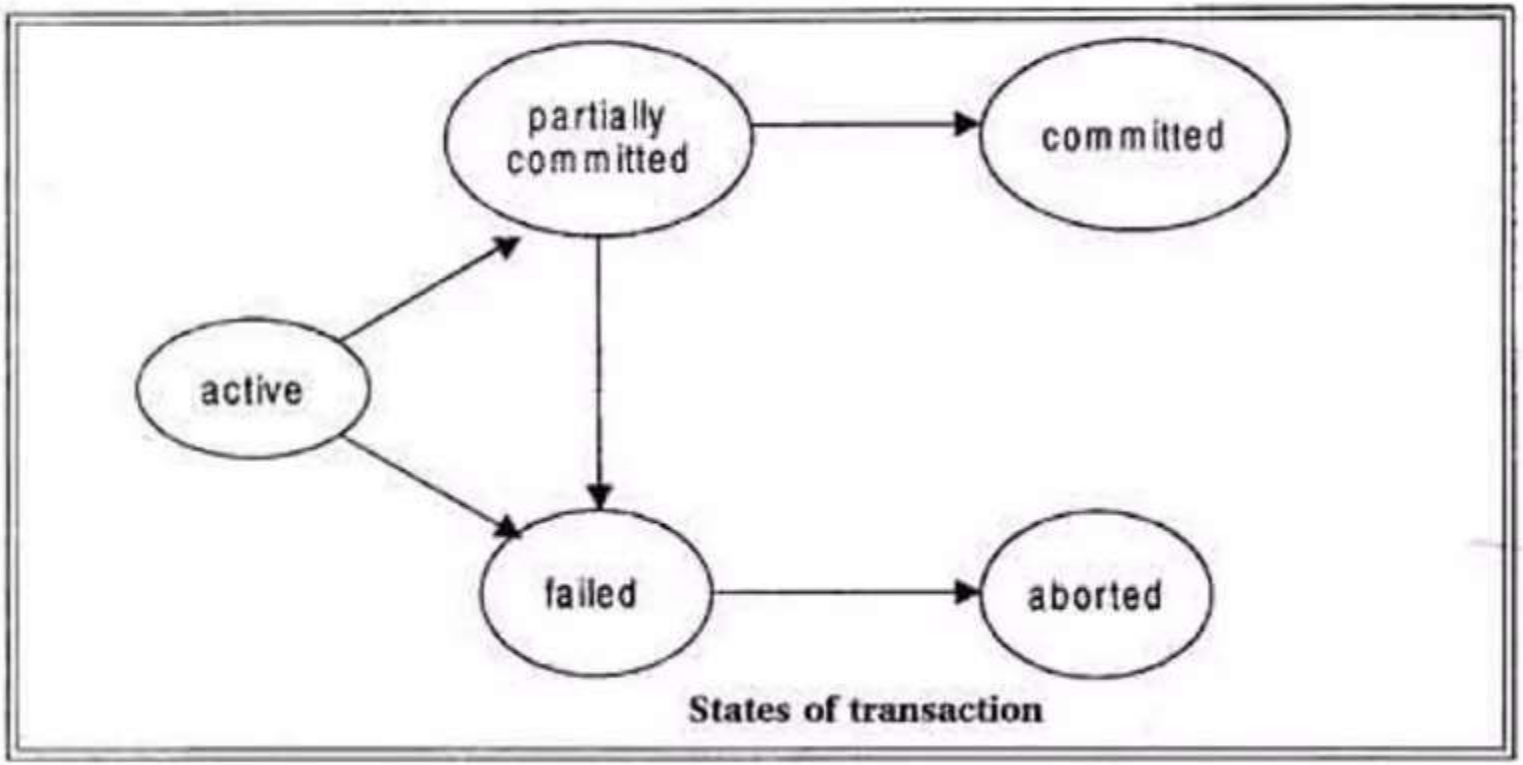
Isolated

- Transactions execute independently of one another
- Database changes not revealed to users until after transaction has completed
- Concurrent transactions cannot interfere with each other

Durable

- Database changes are permanent
- The permanence of the database's consistent state

Transaction State



States of transaction

Transaction State

- A transaction starts in the active state. When it finishes its final statement, it enters the partial committed state.
- When the last statement of the actual output is written out in disk from main memory, the transaction enter the committed sates.
- A transaction enter the failure state after the system determine that the transaction can no longer process with its normal execution. Such a transaction must be roll back.
- Then, it enters the aborted state. At this point the system has two option:

1. It can restart the transaction, if the transaction was aborted as a result of some hardware or software error that was created through the internal logic of the transaction.
2. It can be kill the program, if the transaction was aborted because of internal and logical error that can be corrected only by rewriting the application program.

Transaction Management with SQL

- Transaction support is provided by 2 SQL statements:
  - COMMIT
  - ROLLBACK.
- When a transaction sequence is initiated, it must continue through all succeeding SQL statements until one of the following four events occurs:
  - A COMMIT statement is reached.
  - A ROLLBACK statement is reached.
  - The end of a program is successfully reached (COMMIT).
  - The program is abnormally terminated (ROLLBACK).

Transaction - Example

- Example:

```
UPDATE PRODUCT
SET PROD_QOH = PROD_QOH - 200
WHERE PROD_CODE = 'QS123XY';
UPDATE ACCT_RECEIVABLE
SET ACCT_BALANCE = ACCT_BALANCE
10000
WHERE ACCT_NUM = '12345678';
COMMIT;
```

Transaction - Example

Example of transactions

First SQL statement starts transaction

```
INSERT INTO employee VALUES (100, 'JOHN')
INSERT INTO employee VALUES (200, 'MANDY')
COMMIT
```

empID	name
100	JOHN
200	MANDY

No changes applied due to ROLLBACK

```
DELETE FROM employee WHERE name='MANDY'
UPDATE employee SET empID=101 where name='JOHN'
ROLLBACK
```

empID	name
100	JOHN
200	MANDY

```
UPDATE employee SET name='JACK' where empID=100
COMMIT

ROLLBACK ✖
```

There is nothing to rollback

empID	name
100	JACK
200	MANDY

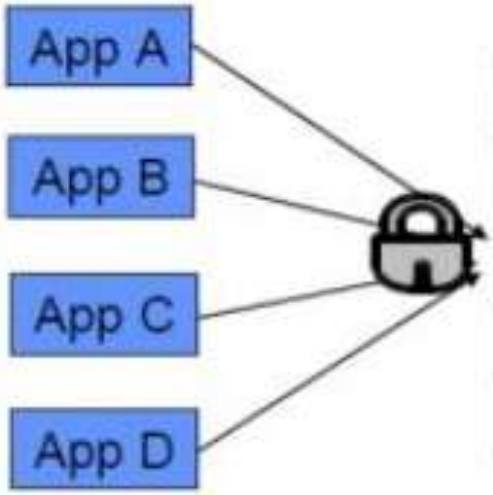
Concurrency Control

- ***Problem*** – in a multi-user environment, simultaneous access to data can result in interference and data loss
- ***Solution***:

Concurrency Control

The process of managing simultaneous operations against a database so that data integrity is maintained and the operations do not interfere with each other in a multi-user environment

Concurrency and Locking

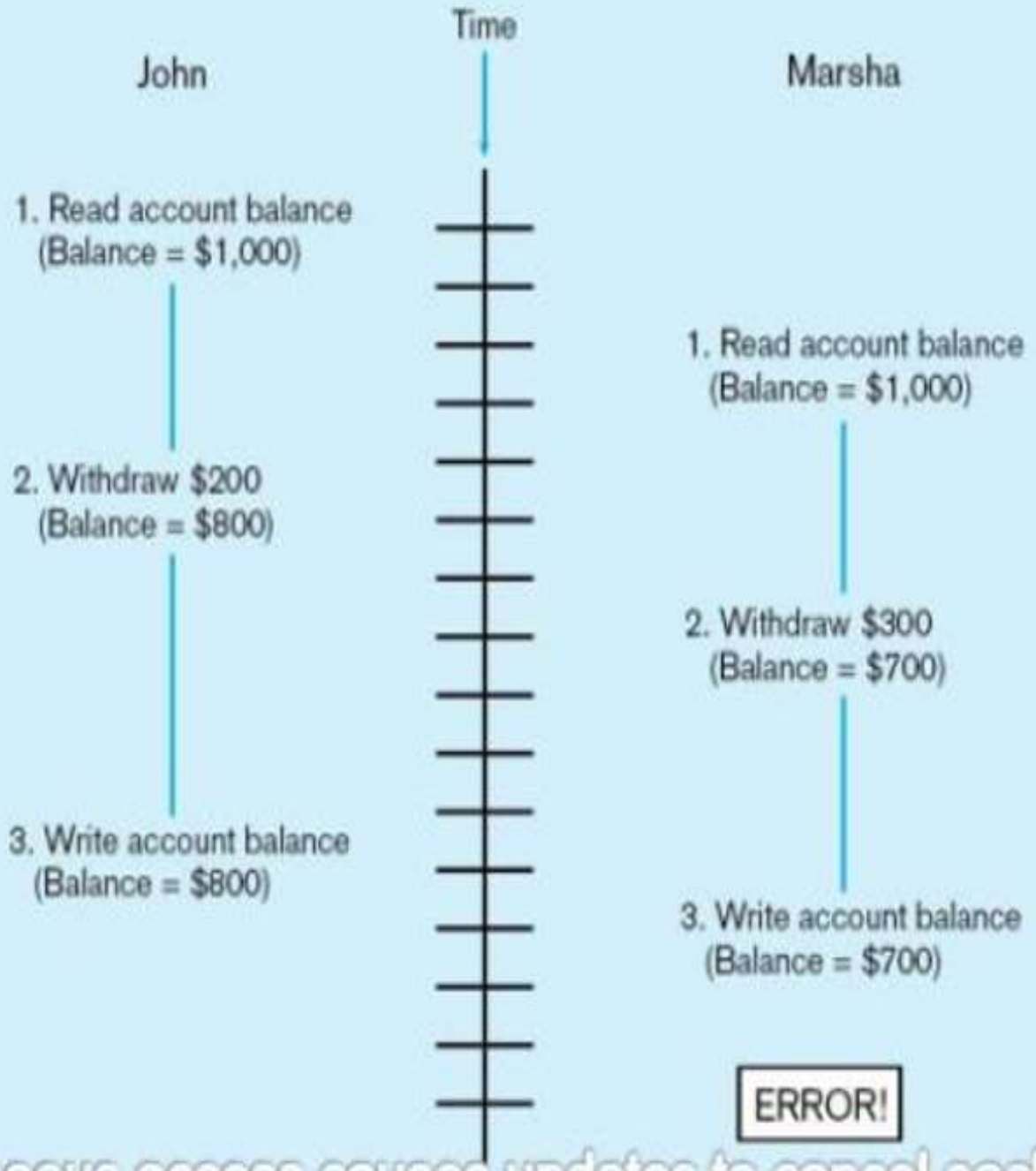


ID	Name	Age
3	Peter	33
5	John	23
22	Mary	22
35	Ann	55

- Concurrency:
  - Multiple users accessing the same resources at the same time
- Locking:
  - Mechanism to ensure data integrity and consistency

Concurrency Control

- The objective of concurrency control is to ensure the serializability of transactions in a multi-user database environment.
- Simultaneous execution of transactions over a shared database can create several data integrity and consistency problems:
  - Lost Updates.
  - Uncommitted Data.
  - Inconsistent retrievals.



Simultaneous access causes updates to cancel each other

Lost Updates - Example

- Using Product Table: Product's quantity on Hand (PROD_QOH)
- Two concurrent transactions update PROD_QOH:

TRANSACTION	COMPUTATION
T1: Purchase 100 units	PROD_QOH = PROD_QOH + 100
T2: Sell 30 units	PROD_QOH = PROD_QOH - 30

- See Table 1 for the serial execution under normal circumstances.
- See Table 2 for the lost update problems resulting from the execution of the second transaction before the first transaction is committed.

Lost Updates - Example

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Read PROD_QOH	35
2	T1	PROD_QOH = 35 + 100	
3	T1	Write PROD_QOH	135
4	T2	Read PROD_QOH	135
5	T2	PROD_QOH = 135 - 30	
6	T2	Write PROD_QOH	105

Exe

Tran

Table 2: Lost Updates

TIME	TRANSACTION	STEP	STORE
1	T1	Read PROD_QOH	
2	T2	Read PROD_QOH	
3	T1	PROD_QOH = 35 + 100	
4	T2	PROD_QOH = 35 - 30	
5	T1	Write PROD_QOH **(Lost update)**	1
6	T2	Write PROD_QOH	

Uncommitted Data

- Occurs when one transaction is allowed to see the intermediate results of another transaction before it has committed.
- Data are not committed when two transactions T1 and T2 are executed concurrently and the first transaction is rolled back after the second transaction has already accessed the uncommitted data - thus violating the isolation property of the transaction.

TRANSACTION	COMPUTATION
T1: Purchase 100 units	PROD_QOH = PROD_QOH + 100 (Rolled bac
T2: Sell 30 units	PROD_QOH = PROD_QOH - 30

Uncommitted Data - Example

Correct Execution Of Two Transactions

TIME	TRANSACTION	STEP	STORED
1	T1	Read PROD_QOH	3
2	T1	PROD_QOH = 35 + 100	
3	T1	Write PROD_QOH	13
4	T1	*****ROLLBACK *****	3
5	T2	Read PROD_QOH	3
6	T2	PROD_QOH = 35 - 30	
7	T2	Write PROD_QOH	

An Uncommi Data Prob

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Read PROD_QOH	35
2	T1	PROD_QOH = 35 + 100	
3	T1	Write PROD_QOH	135
4	T2	Read PROD_QOH (Read uncommitted data)	135
5	T2	PROD_QOH = 135 - 30	
6	T1	***** ROLLBACK *****	35
7	T2	Write PROD_QOH	105

Inconsistent Retrieval

- Occurs when a transaction reads several values the database but a second transaction updates s of them during the execution of the first.
- Occurs when a transaction calculates some sum (aggregate) functions over a set of data while ot transactions are updating the data.
- Example:
  - T1 calculates the total quantity on hand of the proc stored in the PRODUCT table.
  - At the same time, T2 updates the quantity on hand (PROD_QOH) for two of the PRODUCT table's proc

Inconsistent Retrieval - Exa

Retrieval During Update

TRANSACTION T1	TRANSACTION T2
SELECT SUM(PROD_QOH) FROM PRODUCT	UPDATE PRODUCT SET PROD_QOH = WHERE PROD_C
	UPDATE PRODUCT SET PROD_QOH = WHERE PROD_C
	COMMIT;

Inconsistent Retrieval - Examp

	BEFORE	AFTER
PROD_CODE	PROD_QOH	PR
104XCV	100	
110YGH	120	
125TYZ	70	(70 + 30) →
345TYX	35	(35 - 30) →
350TYX	100	
355TYX	30	
Total	**455**	

Transaction Results: Data Entry Correcti

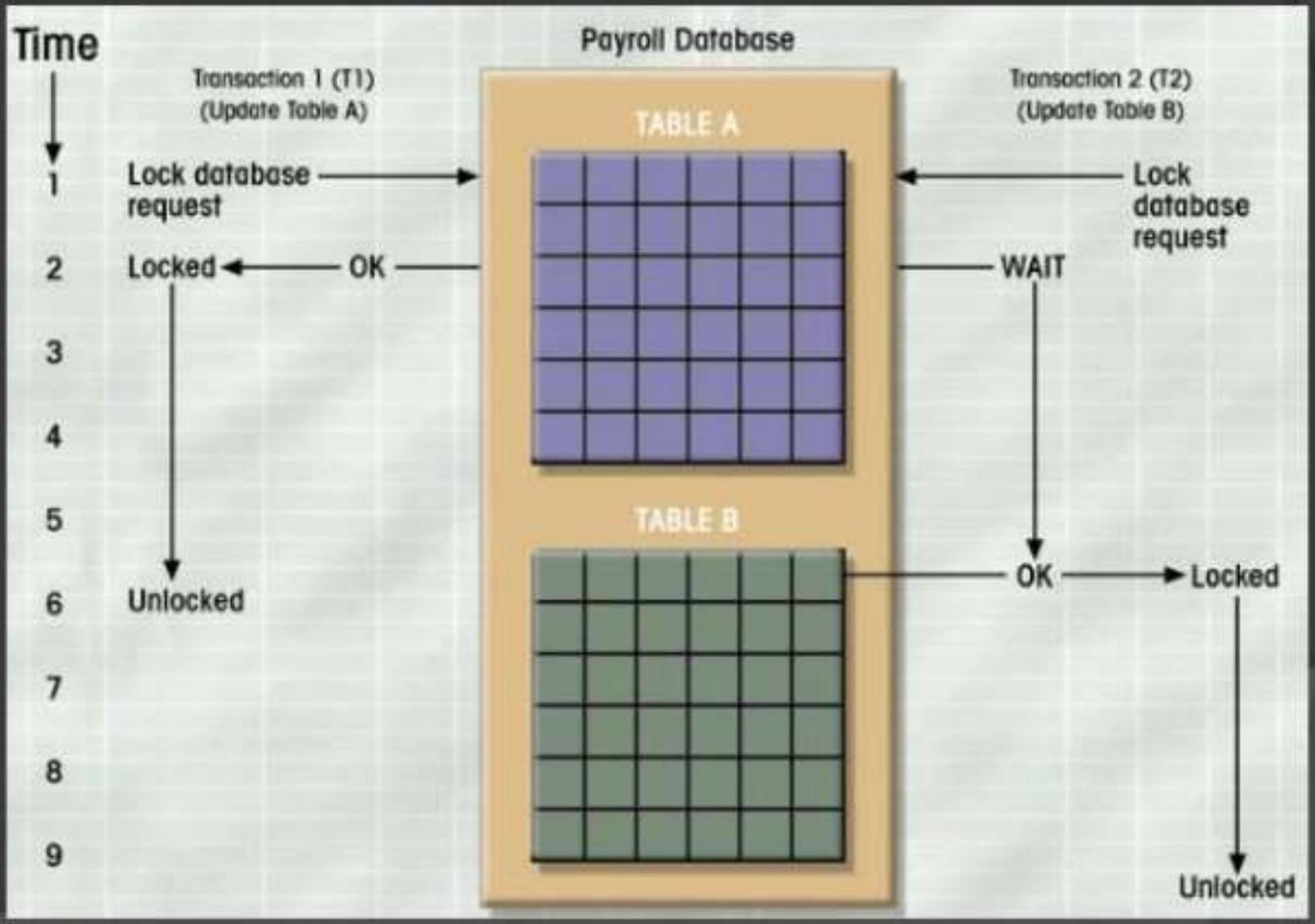
Non-Repeatable Read

- Occurs when a transaction reads the same row of data twice and returns different data values with each read

Concurrency Control Techniques

- Serializability
  - Finish one transaction before starting another
  - Serializable schedule
  - Transactions → not interfere with each other can still run in parallel
- Locking Mechanisms
  - The most common way of achieving serialization
  - Data that is retrieved for the purpose of updating is locked for the updater
  - No other user can perform update until unlocked

Database Lock

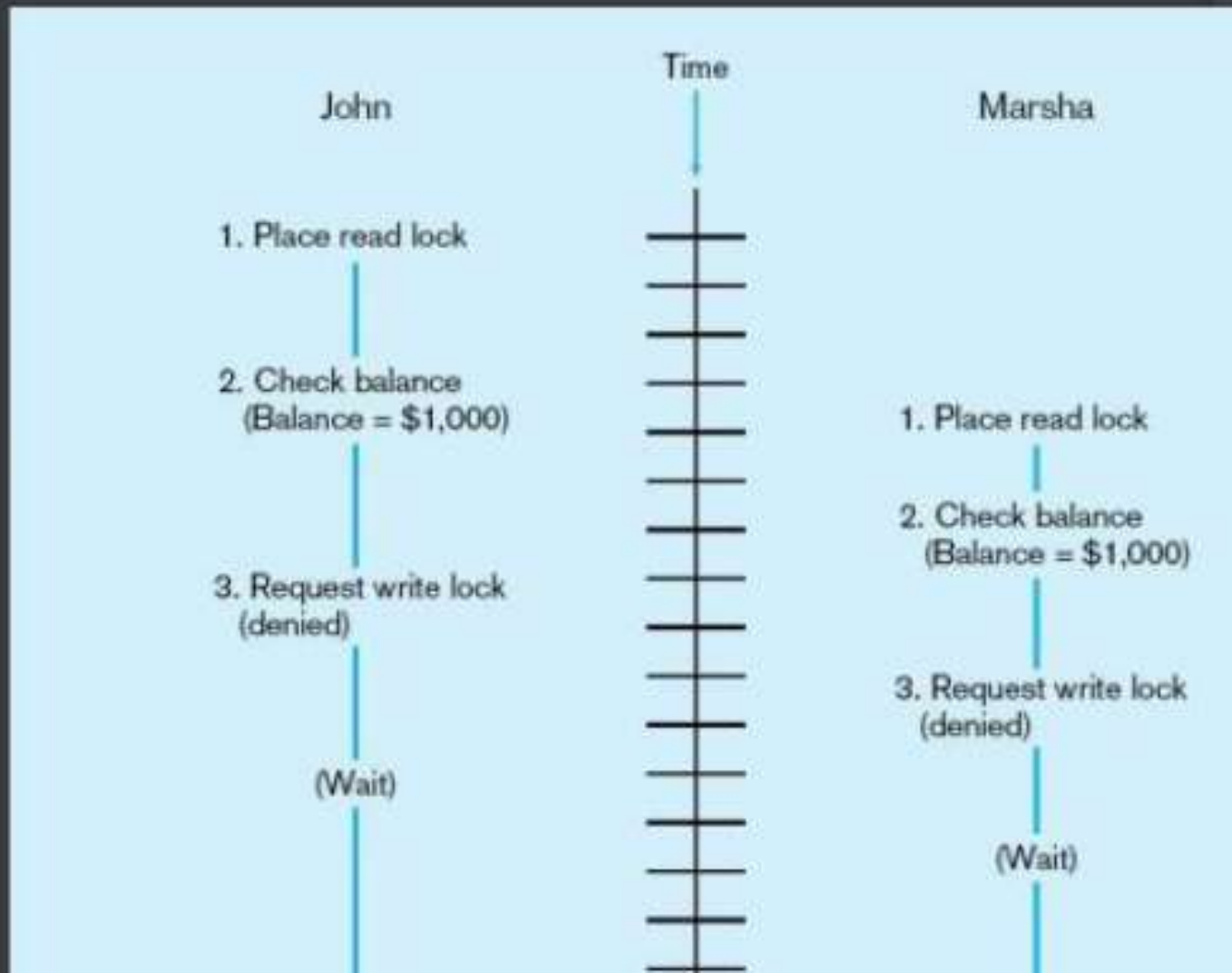


Deadlock

- An impasse that may results when two or more transactions have lo
common resources, and each waits for the other to unlock their reso

The problem of deadlock

John and Marsha will wait forever for each other to release their locked resources!



Managing Deadlock

Downloa

- Deadlock prevention:
  - Lock all records required at the beginning of a transaction
  - Two-phase locking protocol
    - Growing phase
    - Shrinking phase
  - May be difficult to determine all needed resources in advance

Deadlock prevention

- Two-phase locking protocol
  - Growing phase
    - A transaction acquires all the required locks without unlocking any data
    - Once all locks have been acquired, the transaction is in its locked point
  - Shrinking phase
    - A transaction releases all locks and cannot obtain any new lock